

UE : JAVA Processing

Rapport de projet – Jeu Pac-Man en Processing

Etudiante :

IMLOUL Lylia

Encadrant :

Maxime MARIA

Table des matières

1. Introduction 1
2. Architecture du projet 1
3. Fichier de niveau 1
4. Explication des classes 2
5. Déplacement de Pac-Man 5
6. Déplacement et comportement des fantômes 6
7. Gestion des collisions et des scores 6
8. Menu et gestion des états du jeu 7
9. Fin de partie 7
10. Difficultés rencontrées 7
11. Choix techniques et solutions retenues 8
12. Limites du projet et améliorations possibles 8
13. Conclusion 9
14. Biblioraphie 9

1. Introduction

Ce projet consiste en la réalisation d'un jeu Pac-Man en Processing, avec

- Un plateau de jeu défini par un fichier texte (level.txt).
- Des entités : Pac-Man (Hero) et plusieurs fantômes (Ghost).
- Collecte de pac-gommes et super pac-gommes.
- Gestion du score, des vies et états du jeu (pause, victoire, défaite).

L'objectif était de reproduire le comportement classique de Pac-Man avec un déplacement fluide, des collisions correctes et des fantômes.

2. Architecture du projet

Le projet est structuré autour de plusieurs classes principales :

Classe	Rôle
Board	Représente le plateau de jeu, gère la grille, les murs, les pac-gommes et le centrage des cases.
Game	Gestion globale du jeu : score, vies, état du jeu, mise à jour des entités, collisions.
Hero	Pac-Man : gestion du mouvement fluide, collisions avec les murs, collecte des pac-gommes.
Ghost	Fantômes : gestion de la sortie de la cage, déplacement sur la grille, collisions avec Pac-Man.
Menu	Affichage du menu de pause

3. Fichier de niveau

- Le plateau est chargé depuis Level1.txt .
- Première ligne : titre du niveau (donc à ignorer).
- Chaque caractère du fichier représente une cellule :

Caractère	Type de cellule
x	Mur
V	Vide
o	Pac-Gomme
O	Super Pac-Gomme
P	Position initiale de Pac-Man

4. Explication des classes

a. Classe Board

Fonctions principales :

- Type charToTypeCell(char c) :
  - Convertit un caractère du fichier en TypeCell, exemple : ‘x’→ WALL , etc.
- PVector getCellCenter(int i, int j) :
  - Renvoie les coordonnées en pixels du centre de la cellule (i, j).
  - Utile pour centrer Pac-Man et les fantômes.
- Boolean isWall(int x, int y) :
  - Retourne true si la cellule (x, y) est un mur.
  - Vérifie également les limites du plateau.
- TypeCell getCell(int x,int y) :
  - Retourne le type de cellule à la position (x, y).
  - Si hors limites, retourne EMPTY.
- Void setCell(int x, int y, TypeCell type)
  - Modifie le type d’une cellule, par exemple quand Pac-Man mange une pac-gomme.
- PVector findHeroStartPosition()
  - Parcourt le plateau pour trouver la cellule marquée HERO.
  - Retourne la position (x,y) de départ de Pac-Man.
- Boolean hasDotsRemaining()
- Vérifie si des pac-gommes(DOT ou SUPER_DOT) restent sur le plateau.
- Utilisé pour déterminer si le joueur a gagné.
- Void drawIt()
- Dessine le plateau :
  - Murs(WALL) avec rect() ;

- Pac-gommes(DOT) avec ellipse().
- Super pac-gommes (SUPER_DOT) avec ellipse plus grande.

b. Classe Hero

Fonctions principales :

- void launchMove(PVector dir)
- Stocke la direction demandée par le joueur (_nextDirection).
- Void update (Board board)
- Gère le mouvement fluide :
  - Vérifie si proche du centre de la cellule.
  - Change de direction si la direction demandée est possible.
  - Bloque le mouvement si un mur est devant
- Met à jour _position en pixels et _cellX, _cellY en cellule.
- Gère la téléportation horizontale si Pac-Man sort du plateau.
- Void reset(Board board)
- Remet Pac-Man à sa position de départ.
- Réinitialise _direction et _nextDirection.
- Void drawIt()
- Dessine le sprite de Pac-Man centré sur _position .

c. Classe Game

Fonctions principales :

- Void update()
- Met à jour Pac-Man et les fantômes.
- Gère les collisions et la collecte de pac-gommes.
- Met à jour le super mode.

- Vérifie les conditions de fin(victoire / défaite).

➢ Void handleGhostCollision()

- Si collision Pac-Man et fantômes :
    - Si _superMode désactivé : perd une vie.
    - Sinon Pac-Man peut manger le fantôme.

➢ Void drawIt()

- Dessine le plateau, les fantômes, Pac-Man.
- Affiche le menu si pause et l'écran de fin si victoire/défaite.

➢ Void checkCollections()

- Vérifie si Pac-Man est sur une pac-gomme ou super pac-gomme.
- Met à jour le score et active _superMode.

➢ Void updateSuperMode()

- Décrémente _superModeTimer et désactive le super mode quand le temps est écoulé.

➢ Void checkEndConditions()

- Vérifie si toutes les pac-gommes ont été mangées (_won=true).

➢ Void drawUI()

- Affiche le score et le nombre de vies.

➢ Void drawEndScreen()

- Affiche l'écran « GAME OVER » ou « VICTOIRE »

➢ Void handleKey(int k)

- Gère les touches clavier pour :
    - Pause(ESC)
    - Recommencer(R)
    - Quitter(E)
    - Déplacement de Pac-Man(flèches ou ZQSD)

➢ Void restartGame()

- Réinitialise tout : plateau, Pac-Man, fantômes, score, vies et état du jeu.

d. Classe Menu

- Affiche un menu simple au centre de l'écran lors de la pause.

e. Classe Ghost

➢ void update(Board board)

- Si le fantôme est en mode _isInCage, il suit une trajectoire verticale pour sortir. Sinon, il exécute normalMovement.

➢ void normalMovement(Board board) :

- Le fantôme avance toujours tout droit, il regarde si la cage suivante (nextX,nextY) n'est pas un mur. Si c'est un mur, on utilise random(4) et la direction change selon le résultat(0, 1, 2 ou 3).

➢ void updateCellPosition (Board board)

- Elle transforme les coordonnées en pixels.

➢ reset(Board board)

- Quand Pac-Man meurt ou que le jeu recommence, on remet les fantômes à leur point de départ dans la cage.

➢ void drawIt(boolean superMode)

- Gère l'affichage. Si Pac-Man est en super-mode, le fantôme est bleu, sinon il affiche sa couleur normale.

5. Déplacement de Pac-Man

Les déplacements de Pac-Man s'effectuent uniquement le long des couloirs du labyrinthe.

A chaque mise à jour, le jeu vérifie si la direction demandée par le joueur est possible, c'est-à-dire s'il n'y a pas de mur dans la cellule suivante.

Un système de téléportation horizontale est également implémenté : lorsque Pac-Man sort par le bord gauche du plateau, il réapparaît automatiquement à droite, et inversement.

6. Déplacement et comportement des fantômes

Les fantômes commencent la partie dans une cage située au centre du plateau. Ils sortent de cette cage progressivement, chacun après un certain temps d'attente, afin d'éviter qu'ils ne sortent tous en même temps.

Une fois sortis, les fantômes se déplacent de manière pseudo-aléatoire :

- ils avancent tout droit tant que possible ;
- aux intersections, ils peuvent changer de direction de façon aléatoire ;
- ils ne traversent jamais les murs du labyrinthe.

7. Gestion des collisions et des scores

a) Collisions Pac-Man / Pac-gommes

Lorsque Pac-Man passe sur une pac-gomme :

- La pac-gomme disparaît du plateau ;
- Le score du joueur est augmenté de 10 points

Pour une super pac-gomme :

- Le score est augmenté de 50 points ;
- Le mode super est activé pendant une durée limitée.

b) Mode Super Pac-Gomme

Lorsque le mode super est activé :

- les fantômes changent d'apparence(couleur bleue) ;
- Pac-Man peut manger les fantômes.

Lorsqu'un fantôme est mangé :

- il est immédiatement réinitialisé à sa position de départ ;
- le score du joueur est augmenté de 200 points.

A la fin du temps imparti, le mode super est désactivé automatiquement.

c) Collisions Pac-Man / Fantômes

Si Pac-Man entre en collision avec un fantôme hors mode super :

- le joueur perd une vie ;
- Pac-Man et tous les fantômes sont replacés à leur position initiale.

Lorsque le nombre de vies atteint zéro, la partie est perdue.

8. Menu et gestion des états du jeu

Cette partie montre la maîtrise de la gestion des états.

Lorsque le joueur appuie sur la touche echap, le jeu est mis en pause et un menu s'affiche à l'écran.

Pendant la pause, la logique du jeu est complètement arrêtée.

Le menu permet :

- de reprendre la partie ;
- de recommencer une nouvelle partie ;
- de quitter le jeu.

Ce système permet une gestion claire des différents états du jeu : jeu en cours, pause, victoire et défaite.

9. Fin de partie

La partie peut se terminer de deux manières :

- **Victoire** : toutes les pac-gommes du plateau ont été mangées ;
- **Défaite** : le joueur n'a plus de vies disponibles.

Dans les deux cas, un écran de fin s'affiche avec :

- le message « VICTOIRE » ou « GAME OVER » ;
- le score final du joueur.

Le joueur peut alors recommencer une partie à l'aide du clavier.

10. Difficultés rencontrées

Lors du développement du jeu Pac-Man, plusieurs difficultés ont été rencontrées, notamment liées à la gestion des déplacements, des collisions et des états du jeu.

La première difficulté concernait le déplacement fluide de Pac-Man sur une grille. Le plateau est défini sous formes de cellules, mais l'affichage et le mouvement se font en pixels. Il a donc fallu trouver un compromis entre un déplacement continu et un alignement précis sur la grille. La solution adoptée consiste à recalculer

régulièrement la position du héros par rapport au centre des cellules afin d'autoriser les changements de direction uniquement lorque Pac-Man est suffisamment proche du centre.

Une autre difficulté importante concernait la gestion des collisions entre Pac-Man et les fantômes. Afin d'éviter des collisions imprécises en pixels , celles-ci ont été basées sur les coordonnées en cellules. Cela garantit une détection plus simple et plus fiable.

Enfin, la gestion des différents états du jeu(jeu en cours, pause, victoire, défaite) a nécessité une attention particulière pour éviter que la logique du jeu continue à s'exécuter lorsque le jeu est en pause ou terminé.

11. Choix techniques et solutions retenues

Plusieurs choix techniques ont été faits afin de simplifier le développement tout en conservant un comportement proche du jeu original.

Pour le comportement des fantômes, une logique de déplacement pseudo-aléatoire a été choisie. Les fantômes avancent dans une direction tant que cela est possible et changent de direction lorsqu'ils recontrent un mur. Ce choix permet d'obtenir un comportement crédible tout en restant simple à implémenter.

12. Limites du projet et améliorations possibles

Bien que le jeu soit fonctionnel, certaines limites peuvent être identifiées.

Le comportement des fantômes reste relativement simple. Une amélioration possible serait l'implémentation (Dijkstra, A*).

D'autres améliorations pourraient être ajoutées, telles que :

- l'ajout de plusieurs niveaux ;
- l'implémentation de bonus(fruits) ;
- gain de vie à 10 000 points ;
- l'ajout de sons et de musiques ;
- meilleurs scores
- score cumulatif lors de l'enchaînement de fantômes mangés ;
- l'animation de la bouche de Pac-Man ;
- une meilleure gestion graphique des sprites.

Ces améliorations n'ont pas été implémentées par manque de temps, mais elles pourraient constituer des extensions intéressantes du projet.

13. Conclusion

Ce projet m'a permis de concevoir un jeu complet et jouable en appliquant les principes de la programmation orientée objet. L'objectif principal était de structurer un code propre.

Bien que j'aie privilégié la stabilité du moteur de jeu sur l'ajout de fonctionnalités secondaires, le programme constitue une base solide pour être amélioré.

14. Biblioraphie

- Cours JAVA Processing : https://community-sciences.unilim.fr/course/view.php?id=2700
- Processing Reference & Documentation : https://processing.org/
- Wikipedia : https://fr.wikipedia.org/wiki/Pac-Man